

南开大学

《汇编语言与逆向技术》课程实验报告

实验一：HelloWorld



学 院 网络空间安全学院
专 业 物联网工程
学 号 2310422
姓 名 谢小珂
班 级 0410

一、实验目的

- 1、熟悉 Win32 汇编 MASM32 的编译环境；
- 2、命令行输出“HelloWorld”
- 3、窗口输出“HelloWorld”

二、实验原理

1、MASM32

MASM32 是国外的 MASM 爱好者自行整理和编写的一个软件包，最高版本为 11.0 版，MASM32 并不是微软官方发布的软件，微软官方发布的软件 MASM 最新版本也只到 6.15 版，微软发布的 MASM 系列版本从 6.11 版才开始支持 windows 编程，6.11 版以前的版本都不支持 windows 编程，只能用来写 DOS 程序。

MASM32 汇编编译器是 MASM6.0 以上版本中的 ml.exe，资源编译器是 Microsoft Visual Studio 中的 rc.exe，32 位链接器是 Microsoft Visual Studio 中的 Link.exe，同时包含有其他的一些如 lib.exe 和 DumpPe.exe 等工具。

MASM 的 windows 编程的教学书籍有《windows 环境下 32 位汇编语言程序设计第二版》。

2.汇编命令和参数的解析

(1) \masm32\bin\ml /c /Zd /coff hello_console.asm

调用 MASM32 环境中的汇编编译器 ml.exe，对名为 hello_console.asm 的汇编源文件进行编译，生成适用于 Win32 平台的目标文件 (.obj)，并在其中包含调试信息。

各参数解析：

\masm32\bin\ml: 指定 ml.exe 编译器的完整路径，该程序负责将汇编语言源代码转换为机器语言目标文件。

/c: 编译选项，表示"compile only"，即只进行编译操作而不会自动调用链接器。执行后生成.obj 目标文件，但不生成最终的可执行文件。

/Zd: 调试信息选项，指示编译器在目标文件中生成行号调试信息。这些信息可供调试器使用，实现源代码级别的调试跟踪。

/coff: 目标文件格式选项，指定生成 COFF 格式的目标文件。这是 Win32 平台的标准对象文件格式，也是生成 PE 格式可执行文件的前提。

hello_console.asm: 输入参数，指定要编译的汇编源文件名。该文件应包含完整的汇编指令和数据定义。

(2) \masm32\bin\link /SUBSYSTEM:CONSOLE hello_console.obj

调用链接器 **link.exe**，将 **hello_console.obj** 目标文件与相关的库文件进行链接，生成一个控制台子系统的可执行文件。

各参数解析：

\masm32\bin\link: 指定 **link.exe** 链接器的完整路径，该程序负责将目标文件与库文件合并生成最终的可执行文件。

/SUBSYSTEM:CONSOLE: 子系统指定参数，告知操作系统该程序需要在控制台环境中运行。这类程序会自动创建命令行窗口，支持标准输入输出操作。

hello_console.obj: 输入参数，指定要链接的主要目标文件。链接器会解析该文件中的外部引用，并将其与相应的库函数连接。

(3) \masm32\bin\Link /SUBSYSTEM:WINDOWS hello_window.obj

调用链接器 **link.exe**，将 **hello_window.obj** 目标文件链接生成一个 **Windows** 图形界面子系统的可执行程序。

各参数解析：

\masm32\bin\Link: 链接器程序的路径指示，与上一条命令中的 **link.exe** 是同一个程序（注意大小写不敏感）。

/SUBSYSTEM:WINDOWS: 子系统参数，指定生成 Windows GUI 应用程序。这类程序不会自动创建控制台窗口，直接以图形界面方式运行。

hello_window.obj: 输入的目标文件名，该文件应包含窗口程序的相关代码，如 **MessageBox** 等 GUI API 的调用。

3.汇编程序解析：每一行汇编语句的含义

(1) 汇编程序 1: hello_console.asm

该程序是一个 Win32 控制台应用程序，功能是在命令行中输出字符串 "Hello World!"，然后正常退出。使用了 MASM32 提供的 StdOut 函数进行输出，调用 ExitProcess 结束进程。

```
01 .386
02 指定汇编器使用 Intel 80386 及以上处理器的指令集；这是 32 位汇编程序的基础，确保可以使用 32
03 位寄存器和指令。
04 .model flat, stdcall
05 定义程序的内存模型和调用约定；flat 表示使用平坦内存模型，所有段（代码、数据）共享同一个 32
06 位线性地址空间。stdcall 是函数调用约定，参数从右向左压栈，由被调用函数清理堆栈。
07 option casemap :none
08 设置符号大小写敏感；汇编器不会自动转换符号（如函数名、变量名）的大小写，保持源码中的原始写法。
09 include \masm32\include\windows.inc
10 包含 Windows 常量、结构体和函数原型的定义文件；提供 Win32 API 编程所需的基本定义。
11 include \masm32\include\kernel32.inc
12 包含 Kernel32.dll 中函数的声明（如 ExitProcess）；使程序可以调用系统内核函数。
13 include \masm32\include\masm32.inc
14 包含 MASM32 特有函数的声明（如 StdOut）；提供 MASM32 库中的辅助函数。
15 includelib \masm32\lib\kernel32.lib
16 链接时使用 Kernel32.dll 的导入库；使得程序可以正确链接到 ExitProcess 等系统函数。
17 includelib \masm32\lib\masm32.lib
18 链接时使用 MASM32 库的导入库；使得程序可以正确链接到 StdOut 等 MASM32 函数。
19 .data
20 标志数据段的开始；用于存放已初始化的全局变量。
21 str_hello BYTE "Hello World!", 0
22 定义一个以空字符结尾的字符串变量；str_hello 是变量名。BYTE 表示数据类型为字节。"Hello
23 World!", 0 是字符串内容，末尾的 0 表示字符串结束（NULL 终止符）。
24 .code
25 标志代码段的开始；存放程序的可执行指令。
26 start:
27 定义一个标签，作为程序的入口点；链接器会从这里开始执行程序。
28 invoke StdOut, addr str_hello
29 调用 StdOut 函数，输出字符串；invoke 是 MASM 的伪指令，用于调用函数。StdOut 是 MASM32 库中
30 用于控制台输出的函数。addr str_hello 获取字符串 str_hello 的地址作为参数。
    invoke ExitProcess, 0
    调用 ExitProcess 函数，退出程序；ExitProcess 是 Win32 API 函数，用于终止当前进程。0 是退出
    代码，表示程序正常退出。
END start
    标记源程序结束，并指定程序入口点为 start；告诉汇编器和链接器程序从哪里开始执行。
```

(2) 汇编程序 1: hello_window.asm

该程序是一个 Win32 GUI 应用程序，功能是显示一个消息对话框，内容为 "Hello World!"，标题也为 "Hello World!"，只有一个 "确定" 按钮。用户点击确定后，程序正常退出。

与控制台版本的主要区别：

1>使用 user32.inc 和 user32.lib 代替 masm32.inc 和 masm32.lib

2>调用 MessageBox 代替 StdOut 进行输出

3>生成的是窗口程序 (/SUBSYSTEM:WINDOWS) 而非控制台程序

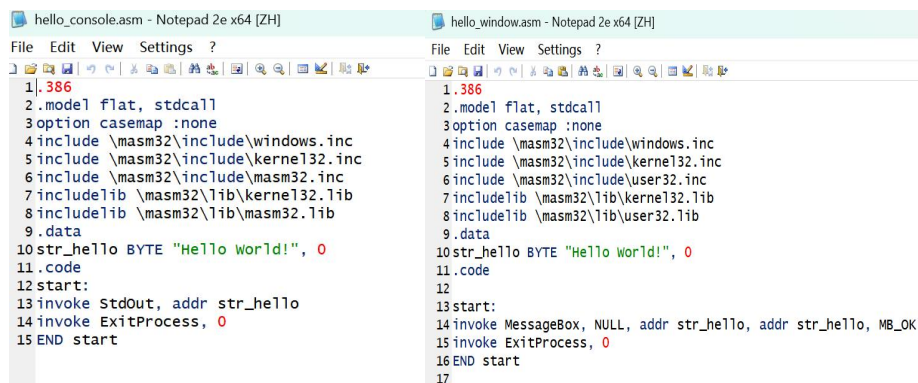
```
01 .386
02 指定使用 Intel 80386 及以上处理器的指令集;为 32 位 Windows 程序提供基础的指令支持
03 .model flat, stdcall
04 定义程序的内存模型和调用约定;flat: 平坦内存模型, 所有段共享 32 位线性地址空间。stdcall: 标准
05 调用约定, 参数从右向左压栈, 被调用函数负责清栈
06 option casemap :none
07 设置符号大小写敏感;保持函数名、变量名的大小写原样, 不进行自动转换。
08 include \masm32\include\windows.inc
09 包含 Windows 常量和结构体定义;提供 Win32 API 编程所需的基本定义, 如 MB_OK 等常量。
10 include \masm32\include\kernel32.inc
11 包含 Kernel32.dll 函数声明;使程序可以调用 ExitProcess 等系统函数。
12 include \masm32\include\user32.inc
13 包含 User32.dll 函数声明;使程序可以调用 MessageBox 等用户界面函数。
14 includelib \masm32\lib\kernel32.lib
15 链接 Kernel32.dll 的导入库;确保 ExitProcess 函数能正确链接。
16 includelib \masm32\lib\user32.lib
17 链接 User32.dll 的导入库;确保 MessageBox 函数能正确链接。
18 .data
19 数据段开始;存放已初始化的全局变量。
20 str_hello BYTE "Hello World!", 0
21 定义以 NULL 结尾的字符串;str_hello: 变量名。BYTE: 字节类型。"Hello World!", 0: 字符串内容,
22 末尾的 0 表示字符串结束。
23 .code
24 代码段开始;存放程序的可执行指令。
25 start:
26 程序入口点标签;链接器从这里开始执行程序。
27 invoke MessageBox, NULL, addr str_hello, addr str_hello, MB_OK
28 调用 MessageBox 函数显示消息框;MessageBox: Windows API 函数, 用于显示消息对话框。NULL: 父
29 窗口句柄(无父窗口)。第一个 addr str_hello: 消息框正文内容。第二个 addr str_hello: 消息框
30 标题。MB_OK: 按钮类型, 只显示"确定"按钮。
    invoke ExitProcess, 0
    调用 ExitProcess 函数退出程序;ExitProcess: 终止当前进程。0: 退出代码, 0 表示正常退出。
```

END start

程序结束标记，指定入口点；告诉汇编器和链接器程序从 **start** 标签开始执行。

三、实验过程

1、**编辑**：用编辑软件（Notepad）形成源程序（.asm），如：hello_console.asm 和 hello_window.asm.



```
hello_console.asm - Notepad 2e x64 [ZH]
File Edit View Settings ?
1|.386
2.model flat, stdcall
3.option casemap :none
4.include \masm32\include\windows.inc
5.include \masm32\include\kernel32.inc
6.include \masm32\include\masm32.inc
7.include lib \masm32\lib\kernel32.lib
8.include lib \masm32\lib\masm32.lib
9.data
10.str_hello BYTE "Hello World!", 0
11.code
12.start:
13.invoke StdOut, addr str_hello
14.invoke ExitProcess, 0
15.END start

hello_window.asm - Notepad 2e x64 [ZH]
File Edit View Settings ?
1|.386
2.model flat, stdcall
3.option casemap :none
4.include \masm32\include\windows.inc
5.include \masm32\include\kernel32.inc
6.include \masm32\include\user32.inc
7.include lib \masm32\lib\kernel32.lib
8.include lib \masm32\lib\user32.lib
9.data
10.str_hello BYTE "Hello World!", 0
11.code
12.start:
13.invoke MessageBox, NULL, addr str_hello, addr str_hello, MB_OK
14.invoke ExitProcess, 0
15.END start
17
```

2、**编译**：用汇编程序（\masm32\bin\ml.exe）对源程序进行汇编，最好打开当前文件所在文件夹的终端：



```
PS K:\大三上\汇编实验\实验报告\实验一> \masm32\bin\ml /c /Zd /coff hello_console.asm
Microsoft (R) Macro Assembler Version 6.14.8444
Copyright (C) Microsoft Corp 1981-1997. All rights reserved.

Assembling: hello_console.asm

*****
ASCII build
*****

PS K:\大三上\汇编实验\实验报告\实验一> \masm32\bin\ml /c /Zd /coff hello_window.asm
Microsoft (R) Macro Assembler Version 6.14.8444
Copyright (C) Microsoft Corp 1981-1997. All rights reserved.

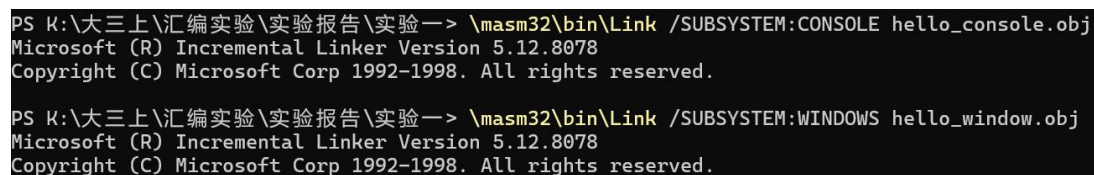
Assembling: hello_window.asm

*****
ASCII build
*****
```

形成目标文件（.obj）：

hello_console.obj
hello_window.obj

3、**链接**：用连接程序（\masm32\bin\link.exe）对目标程序进行链接：



```
PS K:\大三上\汇编实验\实验报告\实验一> \masm32\bin\Link /SUBSYSTEM:CONSOLE hello_console.obj
Microsoft (R) Incremental Linker Version 5.12.8078
Copyright (C) Microsoft Corp 1992-1998. All rights reserved.

PS K:\大三上\汇编实验\实验报告\实验一> \masm32\bin\Link /SUBSYSTEM:WINDOWS hello_window.obj
Microsoft (R) Incremental Linker Version 5.12.8078
Copyright (C) Microsoft Corp 1992-1998. All rights reserved.
```

形成可执行文件（.exe）：

hello_console.exe

hello_window.exe

4、执行：如果结果在屏幕在显示，则直接执行可执行文件。

```
PS K:\大三上\汇编实验\实验报告\实验一> K:\大三上\汇编实验\实验报告\实验一\hello_console.exe
Hello World!
PS K:\大三上\汇编实验\实验报告\实验一> K:\大三上\汇编实验\实验报告\实验一\hello_window.exe
PS K:\大三上\汇编实验\实验报告\实验一>
```



四、实验结论及心得体会

通过这次 HelloWorld 汇编实验，我初步了解了 Win32 汇编程序从编写到运行的完整流程，以及 MASM32 编译环境的工作机制。在实践过程中，我也意识到命令行操作的重要性。最初由于文件路径设置不当，多次出现“无法打开文件”的错误。后来通过理解相对路径与绝对路径的区别，才顺利完成了编译和链接。此外，通过对比控制台输出和窗口输出两种方式，我明确了 SUBSYSTEM 参数的意义，认识到不同子系统对应不同的程序类型和运行环境。这种“所见即所得”的编程体验，让我对计算机系统的执行过程有了更直观的认识。